Pedagogical Tool for Defeasible Reasoning

Nikita Martin
mrtnik003@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Samukelisiwe Zwane
zwnsam011@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Liam De Saldanha
dsllia002@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Abstract

Defeasible reasoning is a form of non-monotonic reasoning that allows conclusions to be revised when new or more specific information is introduced, and has become increasingly important in AI. Despite this, there is currently no dedicated pedagogical tool for defeasible reasoning, as existing pedagogical tools focus exclusively on classical monotonic reasoning, while defeasible reasoners are designed for research use. This project proposes the development of a web-based pedagogical explanation tool that supports three defeasible reasoning algorithms: Rational Closure, Lexicographic Closure, and Relevant Closure, each providing a step-through interface that exposes and explains the internal computational steps of the algorithm. This tool aims to bridge the gap between classical logic education and defeasible reasoning, making defeasible entailment accessible without requiring research-level expertise.

Keywords

Propositional Logic, Defeasible Reasoning, Rational Closure, Lexicographic Closure, Relevant Closure, Pedagogical Tool, Logic Education, Tweety Project

1 Introduction

Humans and machines differ fundamentally in the way they process information and derive conclusions from their knowledge. When given new information, humans intuitively refine their conclusions. However, replicating this behaviour in machines is not simple. Knowledge Representation and Reasoning is an area of Artificial Intelligence that uses symbolic representations to represent different environments and to derive conclusions based on known information [3]. Classical reasoning systems manage and retrieve information from a knowledge base and draw conclusions from it. They adhere strictly to deductive validity. This means that once a conclusion is derived, it cannot be retracted, even when new or contradictory information is introduced. This monotonic nature makes classical reasoning inadequate for modelling the complexity of the real world, where exceptions and evolving circumstances are a part of life.

Defeasible reasoning addresses this limitation by allowing for rational but not necessarily deductively valid conclusions, loosening the rigid structure of classical reasoning. A conclusion is considered defeasible when it is supported by available evidence but remains open to revision if new, contradictory information arises [21]. Defeasible reasoning plays an increasingly important role in AI as intelligent systems grow more reliant on handling incomplete and context-dependent knowledge.

Although some pedagogical tools for classical logic and some defeasible reasoners exist, none bring the two together in an educational context. This creates a clear gap, making it difficult for students to transition from classical to defeasible reasoning without an integral understanding of how a particular conclusion was reached. This project aims to address this gap through the development of a web-based pedagogical tool for defeasible reasoning, providing students with a structured and interactive way to engage with and understand how defeasible entailment is computed.

This proposal is structured as follows. Section 2 provides the theoretical background, introducing classical propositional logic and its limitations, formalising defeasible reasoning through the KLM framework, three defeasible entailment methods and an introduction to existing tools. Sections 3 and 4 present the problem statement and research questions. Section 5 describes the methodology, covering algorithm implementation, the tool and evaluation. Section 6 addresses ethical, professional and legal considerations. Section 7 outlines the anticipated outcomes, and Section 8 presents the project plan.

2 Background

2.1 Classical Logic

Propositional logic is a mathematical formalisation of natural language and a logic framework to model information about the world. A finite set $\mathcal{P} = \{p, r, q, ...\}$ of propositional atoms represents the fundamental statements that can be assigned a value of true or false. Boolean operators $(\neg, \wedge, \vee, \rightarrow, \leftrightarrow)$ are used recursively to define more complex formulas. The set of all formulas over the set $\mathcal{P}$ is the propositional language, $\mathcal{L}$ [2]. The interpretation of this language is given by valuations. A valuation is a function $u : \mathcal{P} \rightarrow \{T, F\}$ that assigns a truth value to every propositional atom in the language, a valuation can be $p\overline{q}$ where p is true and q is false. A true valuation is then denoted $u \Vdash a$, read valuation u satisfies a. This can be extended to any $\alpha, \beta \in \mathcal{L}$ such that a valuation u can be said to satisfy any formula in $\mathcal{L}$ using the criteria:

- $u \Vdash \neg\alpha$ if and only if $u \nVdash \alpha$
- $u \Vdash \alpha \wedge \beta$ if and only if $u \Vdash \alpha$ and also $u \Vdash \beta$
- $u \Vdash \alpha \vee \beta$ if and only if either $u \Vdash \alpha$ or $u \Vdash \beta$
- $u \Vdash \alpha \rightarrow \beta$ if and only if $u \nVdash \alpha$ or $u \Vdash \beta$
- $u \Vdash \alpha \leftrightarrow \beta$ if and only if $u(\alpha) = u(\beta)$

and the set of valuations for $\mathcal{L}$ is denoted as $\mathcal{U}$. Any valuation that satisfies every formula in a finite knowledge base $\mathcal{K}$ is a model of $\mathcal{K}$. From satisfaction, classical entailment allows one to determine if a given statement follows from a knowledge base. A statement α is entailed by a knowledge base $\mathcal{K}$ (denoted as $\mathcal{K} \models \alpha$) when every model of K is also a model of α, capturing the intuition that α is a logical consequence of $\mathcal{K}$[12].

While propositional logic offers a precise and compact framework for representing and evaluating logical relationships, its monotonic nature means that once a conclusion is derived, it cannot be

retracted. If $\mathcal{K} \models \alpha$, then $\mathcal{K} \cup \mathcal{K}' \models \alpha$ for any additional set of formulas $\mathcal{K}'$ [22]. Therefore, once a conclusion is derived, it remains valid regardless of what new information is subsequently added. Although this ensures that inferences are sound, it also becomes a limitation when reasoning must accommodate context-dependent knowledge. Consider the knowledge base:

$$K = \{bird \rightarrow flies, penguin \rightarrow bird, penguin \rightarrow \neg flies\}$$

Using classical reasoning, from $bird \rightarrow flies$ and $penguin \rightarrow bird$, it is possible to derive $penguin \rightarrow flies$. However, K also contains $penguin \rightarrow \neg flies$. Together, $penguin \rightarrow flies$ and $penguin \rightarrow \neg flies$ classically entail $\neg penguin$, that is, classical reasoning concludes that penguins can not exist, which is undesirable, as the intended interpretation is that penguins are an exception to the general rule that birds fly. The real world is inherently complex, with exceptions and evolving circumstances that classical logic cannot adequately model. Therefore, any logic intended to formalise default reasoning must be non-monotonic, allowing for the revision of beliefs when the default assumptions they are based on are invalidated [22].

2.2 Defeasible Reasoning

Defeasible reasoning is a form of non-monotonic reasoning in which conclusions are drawn tentatively and may be revised when new or more specific information is presented [14]. Defeasible reasoning allows for the retraction of conclusions that are *defeated* by incoming evidence; conclusions are considered defeasible when they are supported by available knowledge but lack deductive certainty, remaining open to revision [21].

What distinguishes defeasible reasoning from nonmonotonic reasoning more broadly is the notion of *defeat*, a conclusion is not merely removed, but overridden by more specific or stronger information, while the general default that produced it remains valid for other cases [12, 21].

The KLM framework, proposed by Kraus, Lehmann and Magidor, provides the dominant formal foundation for defeasible reasoning. KLM extends propositional logic, through the notion of a preferential consequence relation, with a defeasible implication operator ($\mid\sim$). This allows for the expression of conditional assertions of the form $\alpha \mid\sim \beta$, where α and β are propositional formulas and reads "typically, if α then β" [18]. Returning to the earlier example, the statement $bird \mid\sim flies$ captures the default that birds typically fly without asserting it universally, allowing the more specific information that penguins do not fly to override this default without creating inconsistency.

A central concept for computing defeasible entailment is materialisation, which converts a defeasible implication $\alpha \mid\sim \beta$ into its classical counterpart $\alpha \rightarrow \beta$. The materialisation of a knowledge base $\mathcal{K}$, denoted $\overrightarrow{\mathcal{K}}$, is the set of all such classical counterparts [12, 14].

This conversion is significant because it grounds defeasible reasoning in classical entailment checking: a formula α is exceptional with respect to $\mathcal{K}$ if $\overrightarrow{\mathcal{K}} \models \neg\alpha$, meaning that assuming α will lead to a contradiction in the materialised knowledge base [5, 12]. Exceptionality is therefore a satisfiability check, which is a problem of determining whether a formula can be satisfied with some valuation. This can be decided by a SAT solver, which is a system that checks the existence of a satisfying valuation for a given formula [19]. Examples of SAT solvers include DPLL and Grasp [6, 16].

2.3 Rational Closure

Rational Closure, first defined by Lehmann and Magidor [14], is the most fundamental form of defeasible entailment. Its central idea is to assign each defeasible statement a base rank reflecting how exceptional it is, where general defaults are assigned low ranks, more specific exceptions are assigned higher ranks, and classical statements are assigned rank ∞ [5, 12]. This ranking is computed by the BaseRank algorithm, which iteratively applies the exceptionality check to $\overrightarrow{\mathcal{K}}$, partitioning statements into ranked sets $R_0, R_1, \ldots, R_{n-1}, R_\infty$ until a fix point is reached [5]. Given these base ranks, the Rational Closure algorithm answers a defeasible query $\alpha \mid\sim \beta$ by continuously removing the lowest ranks until α is no longer exceptional with respect to the remaining set, then checking whether the remaining statements classically entail $\alpha \rightarrow \beta$ [5, 12]. It checks the classical entailment a polynomial number of times in the size of K showing that computing rational closure is no harder than classical entailment [12].

Despite its theoretical appeal, Rational Closure has a limitation known as the drowning problem [1, 12], that is, when a subclass is exceptional with respect to one property, the algorithm discards the entire rank containing the conflict, including statements unrelated to the exception. This conservative behaviour motivates the development of bolder entailment relations.

2.4 Lexicographic Closure

Lexicographic closure addresses the drowning problem by taking a more presumptive approach to conflict resolution. Rather than discarding an entire rank when α is exceptional, it identifies the minimum number of statements from that rank that must be removed to resolve the conflict while still preserving all other defaults. This is formalised through a seriousness ordering on subsets of defaults where a valuation is preferred if it violates fewer and less specific rules [13]. This is achieved through the SubsetRankProp procedure, which replaces each base rank R_i with a finer-grained sequence of sub-ranks representing all possible subsets of R_i of decreasing size, combined as disjunctions [5]. The Lexicographic Closure algorithm then uses the same structure as Rational Closure, but over this more refined ranking, allowing it to keep defaults unrelated to the conflict.

An entailment relation is LM-rational if it satisfies the KLM properties [5]. Lexicographic Closure is LM-rational and strictly extends Rational Closure, meaning that Lexicographic Closure also draws every inference drawn by Rational Closure, but not the other way around [5, 13]. The trade-off of this method is computational, as the construction of sub-ranks is exponential in the size of each base rank, making Lexicographic Closure significantly more expensive than Rational Closure in the worst case [5].

2.5 Relevant Closure

Relevant Closure, proposed by Casini et al. [4], takes a different approach to the drowning problem by focusing solely on those statements that are directly relevant to a particular query. Rather than re-ranking the entire knowledge base, it identifies the specific

defeasible statements responsible for making the query exceptional, using the notion of an α-justification, which is the smallest subset $J \subseteq \mathcal{K}$ whose materialisation classically entails $\neg\alpha$ [4, 9]. The knowledge base is then partitioned into a relevant set R, containing only those statements in a minimal justification and an irrelevant set $R^- = \mathcal{K} \setminus R$. The Relevant Closure algorithm follows a process similar to that of Rational Closure, but only statements in R may be removed; irrelevant statements in R^- are always retained [4].

In practice, Minimal Relevant Closure is preferred over Basic Relevant Closure, as it restricts the relevant set to the lowest- ranked statements in each justification, preventing the relevant set from becoming unnecessarily large [9]. Relevant Closure draws more inferences than Rational Closure in cases of irrelevant exceptions and does so without the exponential overhead of Lexicographic Closure. However, it does not satisfy all KLM properties and is therefore not LM-rational, placing it inferentially between Rational Closure and Lexicographic Closure [5]. Similar to lexicographical closure, relevant closure is more computationally expensive than rational closure. This is due to the classical entailments checks for computing the justifications [4].

Together, the three entailment relations form a spectrum of defeasible inference: from the minimal caution of Rational Closure, to the targeted relevance of Relevant Closure, to the maximal presumptuousness of Lexicographic Closure.

2.6 Existing Tools

A notable example of a pedagogical tool for classical logic is Iltis, which is a web-based interactive system for teaching propositional, modal and first-order logic through a task-based exercise framework. Students receive structured, targeted feedback designed to identify specific misconceptions rather than merely marking answers correct or incorrect [8]. Another relevant pedagogical tool is LogicLearner, developed by Inamder et al [11]. It is a web application that supports undergraduate students in practising propositional logic proofs through an AI-backed step-by-step proof environment with on-demand hints and has been shown to outperform LLM-based alternatives for guided proof solving. Both tools demonstrate that structured, feedback-driven environments meaningfully support student learning of formal logic. However, both are grounded exclusively in classical monotonic reasoning and offer no support for defeasible concepts such as exceptionality, base ranks, or the revision of conclusions under exceptions.

On the defeasible reasoning side, the Tweety Project, developed by Thimm [24] is a collection of Java libraries supporting knowledge representation formalisms. While Tweety demonstrates that defeasible reasoning can be implemented in an expressive system, it is designed for research use rather than education, as interaction with Tweety occurs entirely through Java APIs.

3 Problem Statement

3.1 Broader Problem

Defeasible reasoning has become increasingly important in AI, as it enables systems to handle incomplete knowledge, exceptions and dynamic environments in areas such as knowledge representation and reasoning [12], legal reasoning [7], and medical diagnoses [15]. As logic education plays an important role in Computer Science [8], and AI systems increasingly rely on defeasible reasoning in real-world applications, there is a growing need for students to understand it beyond classical logic [12].

3.2 Limitations of Existing Tools

Despite its growing importance, defeasible reasoning is rarely included in logic education. Existing pedagogical tools like LogicLearner [11] and Iltis [8] focus only on classical monotonic reasoning and do not support defeasible reasoning, while defeasible reasoners like the Tweety Project [24] are meant for research, require programming expertise, lack a graphical user interface, and offer no learning support. To our knowledge, there is no dedicated pedagogical tool for defeasible reasoning, creating a gap between defeasible reasoners and pedagogical tools that makes it difficult for students to bridge their theoretical understanding of classical logic with defeasible reasoning.

3.3 Target Users and Needs

The target users of such a tool are students with a background in propositional logic, who require an accessible and interactive way to engage with defeasible reasoning concepts without needing research-level expertise. These students need a tool that explains the computational steps of defeasible reasoning algorithms in a way that is clear and understandable, bridging their knowledge of classical logic with defeasible reasoning.

4 Research Questions

The lack of a dedicated pedagogical tool for defeasible reasoning and the difficulty students face in bridging theoretical understanding with practical application motivate the following research questions. Through investigating these research questions, we aim to provide meaningful insights into the feasibility of developing a web-based pedagogical explanation tool for KLM-style defeasible reasoning.

(RQ1). To what extent does a web-based interface that exposes the computational steps of the Rational Closure algorithm, including BaseRank construction, exceptionality checking, and entailment verification, accurately and meaningfully represent these steps to students with a background in propositional logic?

(RQ2). How effectively does the step-through visualisation of the Lexicographic Closure illustrate how it resolves the drowning problem identified in Rational Closure, and convey the trade-offs between the two algorithms?

(RQ3). How accurately does the step-through visualisation of Relevant Closure illustrate how justification-based partitioning identifies only the statements responsible for a conflict, and how it differs from the rank-based approaches of Rational Closure and Lexicographic Closure?

This project will be approached incrementally, prioritising the core step-through features before considering additional features. Should time allow, a further question may be investigated depending on the progress made on the core features: to what extent can a Large Language Model API reliably generate accurate and pedagogically useful explanations of the computational steps of the defeasible reasoning algorithms, as evaluated against known outputs produced by the step-through interface? This extension

would only be pursued once the main step-through features of each algorithm are implemented and functional.

5 Methodology

5.1 Overview

The project consists of three phases: algorithm implementation, tool implementation, and evaluation. The first phase will focus on implementing the algorithms for the various closures. The second phase will focus on developing essential features for the custom tool. In the final phase, we evaluate the tool's correctness and usefulness.

5.2 Algorithm Implementation

All three algorithms will be implemented on top of an existing Java logical library, TweetyProject [24]. Time will be taken to familiarise ourselves with the necessary data structures and methods. TweetyProject was chosen for its foundational logical infrastructure and the incorporation of a SAT solver. The TweetyProject provides implementations of rational closure, which will be used for reference; however, our tool will require access to exposed internal reasoning states. An abstract representation of the reasoner will be designed to state all the information and methods a reasoner must expose to the tool. We will also capture the computational states of the algorithms for later tracing. This common interface allows the closure algorithms to integrate seamlessly with the tool.

To implement our reasoners, we will revise how the algorithms for each closure work. Previous projects have focused on optimising algorithms for rational and lexicographical closure [20, 25]. We will study how these projects implemented their solutions and understand how we will expose the details to the tool.

Rational closure would be implemented in accordance with the KLM preferential framework. The implementation would expose the baserank construction and rational closure algorithm. Lexicographical closure would refine rational closure, exposing the subset construction and its ordering. Relevant closure would expose the construction of the relevant and irrelevant subsets.

Challenges : When attempting to use an optimised algorithm, we could encounter that its implementation obscures the intermediate information we need to expose to the tool. Optimisations may combine multiple logical steps into one.

5.3 Explanation tool

The tool will be implemented as a web-based tool using a Spring Boot and React stack. The decision to use a web-based tool is because of the accessibility and visualisation that React components offer. The user will interact with the tool through a GUI where they can perform essential actions such as populating a knowledge base and inserting queries.

The backend will expose the reasoners via a RESTful API. When the frontend calls the API, a JSON object will be returned with the necessary data to populate the frontend and enable the user to understand the logical decisions of the algorithms.

Previous projects have designed similar debugging systems; time will be taken to study their system design and approach [17]. There are obvious extensions, such as optimised algorithms and the addition of the relevant closure algorithm. Beyond these extensions, our tool will allow users to step through each computational state in the algorithm for an entailment in a visual manner. Each computational state will include justification for why it was done, and attached is the context in plain English with a formal reason. Each algorithm would have varying implementations on how the step-through execution would operate and the needed justification.

Challenges: Designing the frontend to communicate logical decisions effectively is hard. We need to create an intuitive tool that users can follow.

5.4 Evaluation

The evaluation of this project will be conducted across two dimensions: correctness and pedagogical effectiveness.

The correctness of each algorithm implementation will be verified by comparing its output against the existing reference implementations. Validation datasets will be generated using knowledge-base generators [10, 23]. Regression testing, using verified examples of defeasible entailment, will be designed to ensure that correctness is maintained if changes and optimisations are made later in development. Integration tests will be used to verify all backend endpoints and ensure that the frontend and backend exchange the necessary information correctly. This ensures varied datasets are represented correctly.

The usability and pedagogical effectiveness of the tool will be assessed by the supervisor and members of the Logic-Based AI research group at the University of Cape Town, led by the supervisor. The tool will be assessed on usability in the context of learnability, ease of use and accuracy. Sessions will be scheduled with members for in-person assessment and feedback. In the event members aren't available an alternative would be to communicate via email.

The feedback from multiple researchers with expertise in defeasible reasoning provides insight into whether the tool successfully explains the computational steps of each algorithm in a way that is clear, understandable, and accessible for users. Through the incremental improvement and feedback loop, we hope to design a tool that succeeds in teaching students defeasible reasoning.

Challenges: The tool features are correct, but not effective for pedagogical use.

5.5 System Architecture

Users interact with the tool through a React frontend where they can populate a knowledge base and submit a defeasible query. The query triggers an API call to the Spring Boot backend, where the custom closure reasoners execute the selected algorithm, capturing each discrete computational state as a trace step. The full trace is returned as a JSON object for the frontend to render as a step-through debugging view. The custom reasoners are built on top of the TweetyProject, using its existing data structures and SAT solver for classical entailment checks. The option to save knowledge bases in a Supabase PostgreSQL database will also be considered.

6 Ethical, Professional, and Legal Issues

This project involves the development of a software tool and does not involve the collection of personal data. Users interact with the tool by inputting knowledge bases and queries, none of which are stored or transmitted beyond the scope of the session. As a result, no formal ethics clearance is required for this project.

This project makes use of open-source libraries, notably the Tweety Project [24], which will be used in accordance with their respective licensing terms. All referenced work will be cited to maintain academic integrity, and the project will maintain professional software development standards, including version control, documentation, and testing.

If an AI-assisted chatbot is implemented, the tool will utilise a Large Language Model API. Since this tool is intended for educational purposes, carefully engineered prompts will constrain the model's responses to the specific reasoning steps being displayed, reducing the risk of misleading explanations being presented to the users.

7 Anticipated Outcomes

7.1 Functional System

The primary outcome of this project will be the delivery of a fully functional prototype of a web-based pedagogical explanation tool comprising three individual algorithm components: Rational Closure, Lexicographic Closure, and Relevant Closure. Each component will provide a step-through interface that exposes the internal computational steps of the algorithm to the user.

7.2 Educational Impact

This project investigates whether a dedicated pedagogical tool can effectively address the gap between defeasible reasoners and logic education by clarifying and making the reasoning processes behind defeasible entailment more understandable to students with a propositional logic background.

7.3 Success Metrics

The success of this tool will be evaluated using the following measurable indicators:

(1) Algorithm correctness: Unit tests against verified examples of defeasible entailment.
(2) Backend correctness: Integration tests to confirm that all backend API endpoints function correctly.
(3) Pedagogical effectiveness: Assessments by the supervisor and Logic-Based AI research group at UCT, evaluating whether the explanations of each algorithm are clear and understandable for students.
(4) Accessibility: Usable by students with a propositional logic background without requiring additional expertise.

8 Project Plan

8.1 Risks and Risk Management Strategies

A summary of these risks and their management strategies is provided in Appendix 9.

8.2 Timeline and Milestones

A Gantt chart is presented in Appendix 9, representing the planned project timeline from May to October 2026, and the key milestones are summarised in Appendix 9. The project will adopt an iterative approach, prioritising core features first, with the chatbot extension pursued only if time allows.

8.3 Resources Required

The backend will be implemented in Java, using the Tweety Project library for classical entailment checking. The frontend will be implemented using React and Spring Boot. Git will be used for version control, and VS Code and IntelliJ will serve as the primary development environments.

Standard laptops and desktops are sufficient for developing and running the tool, as the computational requirements of the Rational Closure, Lexicographic Closure, and Relevant Closure algorithms are no more demanding than classical entailment checking, which is well within the capacity of standard hardware.

8.4 Deliverables

The schedule of deliverables is outlined in Appendix 9.

8.5 Contributions

This project is a collaborative effort, where each member is responsible for investigating an individual research question, through the implementation of a dedicated algorithm component. While the three algorithms differ in complexity, the workload is balanced across the team. Nikita's additional responsibility of implementing and refining the Rational Closure algorithm, which is the most foundational, compensates for the relative simplicity of the Rational Closure algorithm itself against the Lexicographic and Relevant Closure algorithms.

Once the BaseRank foundation is complete, each member will independently implement their respective closure algorithm. Should the workload prove uneven during implementation, the tasks will be redistributed accordingly, with guidance from the supervisor. The work allocation is summarised in Appendix 9.

9 AI Usage Declaration

In this proposal, Claude.ai and Grammarly were used for proofreading, grammar corrections, and clarity improvements on the written sections. The authors produced all content, ideas, design decisions, and academic arguments. The final submission reflects the authors' understanding and judgement.

References

[1] Nick Bassiliades, Grigoris Antoniou, and Ioannis Vlahavas. 2004. DR-DEVICE: A Defeasible Logic System for the Semantic Web. In *Principles and Practice of Semantic Web Reasoning*, Hans Jürgen Ohlbach and Sebastian Schaffert (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 134–148.
[2] Mordechai Ben-Ari. 2012. *Mathematical Logic for Computer Science.* Springer, New York.
[3] R.J. Brachman and H.J. Levesque. 2004. *Knowledge Representation and Reasoning.* Morgan Kaufmann, San Francisco, CA.
[4] G. Casini, T. Meyer, K. Moodley, and R. Nortjé. 2014. Relevant closure: A new form of defeasible reasoning for description logics. *Logics in Artificial Intelligence, Fermé, E. and Leite, J. (eds.)* (2014).
[5] G. Casini, T. Meyer, and I. Varzinczak. 2018. Defeasible entailment: From rational closure to lexicographic closure and beyond. *Proceedings of the 17th International Workshop on Non-Monotonic Reasoning* (2018).
[6] Martin Davis, George Logemann, and Donald Loveland. 1962. A machine program for theorem-proving. *Commun. ACM* 5, 7 (July 1962), 394–397. doi:10.1145/368273.368557
[7] Sandra K. Dewitz, Young Ryu, and Ronald M. Lee. 1994. Defeasible reasoning in law. *Decision Support Systems* 11, 2 (1994), 133–155. doi:10.1016/0167-9236(94)90029-9
[8] Gaetano Geck, Artur Ljulin, Sebastian Peter, Jonas Schmidt, Fabian Vehlken, and Thomas Zeume. 2018. Introduction to Iltis: an interactive, web-based system for teaching logic. In *Proceedings of the 23rd Annual ACM Conference on Innovation*

and Technology in Computer Science Education (Larnaca, Cyprus) *(ITiCSE 2018)*. Association for Computing Machinery, New York, NY, USA, 141–146. doi:10.1145/3197091.3197095

[9] C. Hamayobe. 2026. *Implementation and Evaluation of KLM-Style Defeasible Entailment and Explanation Algorithms*. Master's thesis. Department of Computer Science, Faculty of Science, University of Cape Town.

[10] Joel Hamilton, Joonsoo Park, Aidan Bailey, and Thomas Meyer. 2022. An Investigation into the Scalability of Defeasible Reasoning Algorithms. In *Second Southern African Conference for Artificial Intelligence*. SACAIR 2021 Organising Committee, Online. https://2021.sacair.org.za/proceedings/ Section: 235-251.

[11] Amogh Inamdar, Uzay Macar, Michel Vazirani, Michael Tarnow, Zarina Mustapha, Natalia Dittren, Sam Sadeh, Nakul Verma, and Ansaf Salleb-Aouissi. 2025. LogicLearner: A Tool for the Guided Practice of Propositional Logic Proofs. arXiv:2503.19280 [cs.DM] https://arxiv.org/abs/2503.19280

[12] A. Kaliski. 2020. *An Overview of KLM-Style Defeasible Entailment*. Master's thesis. University of Cape Town, Cape Town, South Africa. http://hdl.handle.net/11427/32743

[13] D. Lehmann. 1995. Another perspective on default reasoning. *Annals of Mathematics and Artificial Intelligence* (1995).

[14] D. Lehmann and M. Magidor. 1992. What does a conditional knowledge base entail? *Artificial Intelligence* (1992).

[15] Luca Longo and Pierpaolo Dondio. 2014. Defeasible Reasoning and Argument-Based Systems in Medical Fields: An Informal Overview. In *2014 IEEE 27th International Symposium on Computer-Based Medical Systems*. 376–381. doi:10.1109/CBMS.2014.126

[16] J.P. Marques-Silva and K.A. Sakallah. 1999. GRASP: a search algorithm for propositional satisfiability. *IEEE Trans. Comput.* 48, 5 (May 1999), 506–521. doi:10.1109/12.769433

[17] Thabo Vincent Moloi. [n. d.]. Extending Defeasible Reasoning Beyond Rational Closure. ([n. d.]).

[18] K. Moodley, T. Meyer, and I.J. Varzinczak. 2012. A Defeasible Reasoning Approach for Description Logic Ontologies. *SA Institute for Computer Scientists and Information Technologists (SAICSIT) Conference* (2012).

[19] Daniel Park. [n. d.]. Scalable Defeasible Reasoning. ([n. d.]).

[20] Evashna Pillay. [n. d.]. An Investigation into the Scalability of Rational Closure V2. ([n. d.]).

[21] John L. Pollock. 1987. Defeasible reasoning. *Cognitive Science* 11, 4 (1987), 481–518. doi:10.1016/S0364-0213(87)80017-4

[22] R. Reiter. 1980. A logic for default reasoning. *Artificial Intelligence* 13 (1980).

[23] Mamodike Sadiki. [n. d.]. A Study of Parameters and Optimization Strategies in Defeasible Knowledge Base Generation. ([n. d.]).

[24] Matthias Thimm. 2017. The Tweety Library Collection for Logical Aspects of Artificial Intelligence and Knowledge Representation. *KI - Künstliche Intelligenz* 31, 1 (March 2017), 93–97. doi:10.1007/s13218-016-0458-4

[25] Dhiresh Thakor Vallabh. [n. d.]. CS/IT Honours Project Final Paper 2022. ([n. d.]).

Appendix A - Milestones

Milestone	Target Date
Planning & Setup (Apr - May)	
Project Proposal Full Draft	24 Apr 2026
Project Proposal Presentations	28-30 Apr 2026
Project Proposal Final Submission	05 May 2026
Algorithm optimisations researched	12 May 2026
Reasoner interface designed	18 May 2026
System architecture finalised	26 May 2026
GUI conceptualised	02 Jun 2026
Development Cycle 1 (Jun - Jul)	
Implement Shared reasoner	09 Jun 2026
Implement Rational Closure reasoner	26 Jun 2026
Implement Lexicographic Closure reasoner	26 Jun 2026
Implement Relevant Closure reasoner	26 Jun 2026
Functional Prototype	05 Jul 2026
First Demo	27 Jul 2026
Development Cycle 2 (Jul - Aug)	
Backend Spring Boot API complete	15 Jul 2026
Frontend UI complete	04 Aug 2026
Frontend and backend integrated	07 Aug 2026
Project Paper Scaffold	10 Aug 2026
Unit and integration testing complete	12 Aug 2026
Full Draft of Final Paper	24 Aug 2026
Possible Extension (Time Permitting)	
AI chatbot interface designed	TBC
AI chatbot interface integrated	TBC
Submission & Finalisation	
Final Paper Submission	04 Sep 2026
Final Code Submission	14 Sep 2026
Final Project Demonstration/Presentation	16 - 23 Sep 2026
Poster Due	28 Sep 2026
Website Due	05 Oct 2026
School of IT Showcase	26 Oct 2026

Appendix B - Project Deliverables

Deliverable	Due Date
Literature Review Scaffold	13 February 2026
Literature Review Due	02 March 2026
Project Proposal Full Draft	24 April
Project Proposal Presentations	28 – 30 April
Project Proposal Final Submission	5 May
Functional Prototype	TBC
Project Progress Demonstration	27 July
Project Paper Scaffold	10 August 2026
Full Draft of Final Paper	24 August 2026
Final Paper Submission	31 August – 5 September
Final Code Submission	04 September 2026
Final Project Demonstration	14 September 2026
Final Project Presentation	16-23 September 2026
Poster Due	28 September 2026
Website Due	05 October 2026
School of IT Showcase	26 October (TBC)

Appendix C - Work Allocation

Member	Responsibilities
All Members	• High-level implementation of the Rational Closure algorithm, more specifically, the BaseRank algorithm, which serves as the foundation for all three individual implementations. • A shared reasoner interface that exposes the internal computational states of the tool • Develop front-end interface into which all three components are integrated. • System integration and testing
Nikita Martin	• Responsible for investigating **RQ1** through the Rational Closure component. • Implement the Rational Closure algorithm, refine and optimize it with enhanced pedagogical depth. • Build a step-through interface that exposes the BaseRank construction, exceptionality checking and entailment verification.
Samukelisiwe Zwane	• Responsible for investigating **RQ2** through the Lexicographic Closure component. • Building on the shared BaseRank, implement the subset ranking procedure and Lexicographic Closure entailment. • Develop the step-through interface that exposes how the drowning problem is resolved through the finer-grained ranking.
Liam De Saldanha	• Responsible for investigating **RQ3** through the Relevant Closure component. • Building on the shared BaseRank, implement the justification computation and partitioning will be implemented • Develop the step-through interface that exposes how relevance-based conflict resolution differs from the rank-based approaches.

Appendix D - Risks and Risk Management Strategies

Table 1: Risk matrix

		Consequences		
		Minor	**Moderate**	**Major**
Probability	**Probable**		R1	
	Possible	R3, R7	R2, R6	R4
	Unlikely			R5

Key: Low risk Medium risk High risk

R1 Changing project requirements **R2** Poor UX design of the tool **R3** Balancing project work with academic workload
R4 A team member is unable to continue **R5** Delayed implementation of the shared Rational Closure foundation
R6 Difficulty understanding the theoretical aspects of the project **R7** Conflict on how to design the final implementation

Risk	Cause	Trigger	Mitigation	Contingency	Monitoring
Changing project requirements	Evolving understanding of scope or supervisor feedback that redirects the focus of the tool.	A component is found to be out of scope or inconsistent with the initial project aims after implementation has begun.	Maintain a clearly defined project scope from the beginning and confirm any changes with the supervisor before implementation begins.	Reassess the project scope and adjust remaining work and prioritise core deliverables.	Review aims and scope at each supervisor meeting to ensure alignment of expectations.
Poor UX design of the tool	Insufficient consideration of the user experience when designing the step-through interface.	Feedback indicating that the tool is difficult to follow or does not communicate reasoning steps clearly.	Consult and incorporate feedback iteratively before finalising the interface.	Redesign components based on feedback received, prioritising clarity of the step-through explanations.	Review interface designs with the supervisor during check-in meetings and gather feedback.
Balancing project work with academic workload	Meeting deadlines across multiple courses alongside project deliverables.	A team member falls behind on project tasks due to pressure from other courses.	Plan the project timeline around known academic deadlines and distribute work evenly to avoid bottlenecks.	Temporarily reduce the scope of additional features and focus on the core system.	Check in as a team weekly to evaluate progress and flag workload concerns early.
A team member is unable to continue	Unforeseen personal, medical, or academic circumstances that prevent a team member from completing the project.	A team member is consistently unable to meet agreed milestones or formally withdraws from the course.	Ensure openness between team members and the supervisor, splitting project into independent parts and keeping detailed documentation so any component can be picked up by another team member if possible.	The project is structured in parallel, so the scope can be reduced by one entailment method without compromising the core deliverable.	Regularly check in with team members about their well-being and progress.
Difficulty understanding the theoretical aspects of the project	Insufficient familiarity with the KLM framework, the three closure algorithms or the foundations of defeasible reasoning.	A team member is unable to implement their assigned component correctly due to gaps in theoretical understanding.	Thoroughly review the relevant literature before implementation begins and schedule dedicated time for understanding before doing implementation work.	Consult with the supervisor for guidance on the theoretical concepts causing difficulty.	Raise questions at meetings with supervisor and keep shared notes to consolidate the team's understanding.
Conflict on how to design the final implementation	Differing opinions between team members on architectural or design decisions for the tool.	A design decision cannot be reached and is blocking progress.	Ensure a collaborative design process where contributions are made and agreed on before implementation begins.	Mediation and guidance from supervisor.	Confirm that all team members are aligned on design decisions before moving to the next implementation phase.

Appendix E - Gantt Chart

